

《计算机程序的构造和解释》期中考试（**参考答案和评分标准**）

(2022-2023 秋季学期, 授课教师: 冯新宇、李樾, 考试方式: 半开卷, 只允许携带一张 A4 纸资料)

学号: _____ 姓名: _____ 院系: _____ 得分: _____ (满分 100)

题目 得分	1	2	3	4	5	6	Extra

注意事项 (请仔细阅读):

1. 试题的难度不是线性递增的, **注意时间分配**。在这项测验中, 可能有些题目较难, 因此你不要在一道题上思考时间太久, 遇到不会答的题目可先跳过去, 如果有时间再去思考。否则, 你可能没有时间完成后面的题目。
2. 代码填空题的**正确答案不止一种**。代码填空题答案正确即可得分, 错误也不会倒扣分。每空**最多只能填一个**Python 语句或表达式, 可以直接使用的 Python 内置函数 (Python Built-in Functions) 包括: `all`, `any`, `bool`, `dict`, `enumerate`, `filter`, `float`, `int`, `iter`, `len`, `list`, `map`, `max`, `min`, `pow`, `print`, `range`, `round`, `sorted`, `sum`, `zip`。

1 What Would Python Display? (14pts)

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is valued. The output may have multiple lines. Each expression has at least one line of output.

- If an error occurs, write **Error**, but include all output displayed before the error.
- To display a function value, write **Function**. To display an iterator value, write **Iterator**.
- If an expression would take forever to evaluate, write **Forever**.

The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`. Assume that you have first started python3 and executed the statements on the left.

```
1 def stay_up_late(todos, hours_per_day=24):
2     sleep_time = 8
3     def finish_one_by_one(todos, remain_time):
4         if remain_time < sleep_time:
5             return True
6         if not todos and remain_time >= sleep_time:
7             return False
8         else:
9             todo, todos = todos[0], todos[1:]
10            return stay_up_late(todos, remain_time - sum(todo))
11    return finish_one_by_one(todos, hours_per_day)
12
13 def time_sequence(todos, rest):
14     for todo in todos:
15         yield sum(todo) + rest
16
17 study = [1, 2, 3, 4]
18 relax = [1] * 4
19 games = [1, 4]
20 meals = [1] + [1] + [1]
21 todos = [study, relax, games, meals]
```

评分标准：第三空 2 分，其他每空 3 分

Expression	Interactive Output
<code>pow(2, 11) - 26</code>	2022
<code>print(4, 5) + 1</code>	4 5 Error
<code>f1 = print(2+3)</code> <code>f2 = lambda: print(2+3)</code> <code>print(f1, f2, f2())</code>	5; 5; None Function None
<code>print(sum(study))</code> <code>print(len(meals))</code> <code>print(len(todos))</code>	10; 3; 4
<code>dropped = True and ((True or False) or True)</code> <code>good_night = dropped or not stay_up_late</code> <code>print(dropped, good_night)</code>	True True
<code>print(stay_up_late(todos))</code> <code>print(stay_up_late(todos, 48))</code> <code>print(stay_up_late([meals]))</code>	True; False; False
<code>tiktok = time_sequence(todos, 0)</code> <code>for todo in tiktok:</code> <code>if todo >= 3:</code> <code>print(next(tiktok))</code> <code>else:</code> <code>print(todo)</code> <code>alltime = 0</code> <code>for todo in tiktok:</code> <code>alltime += todo</code> <code>print(alltime)</code>	4; 3; 0

2 Environment Diagram (14pts)

Fill in the environment diagram that results from executing the code on the right until the entire program is finished, an error occurs, or all frames are filled.

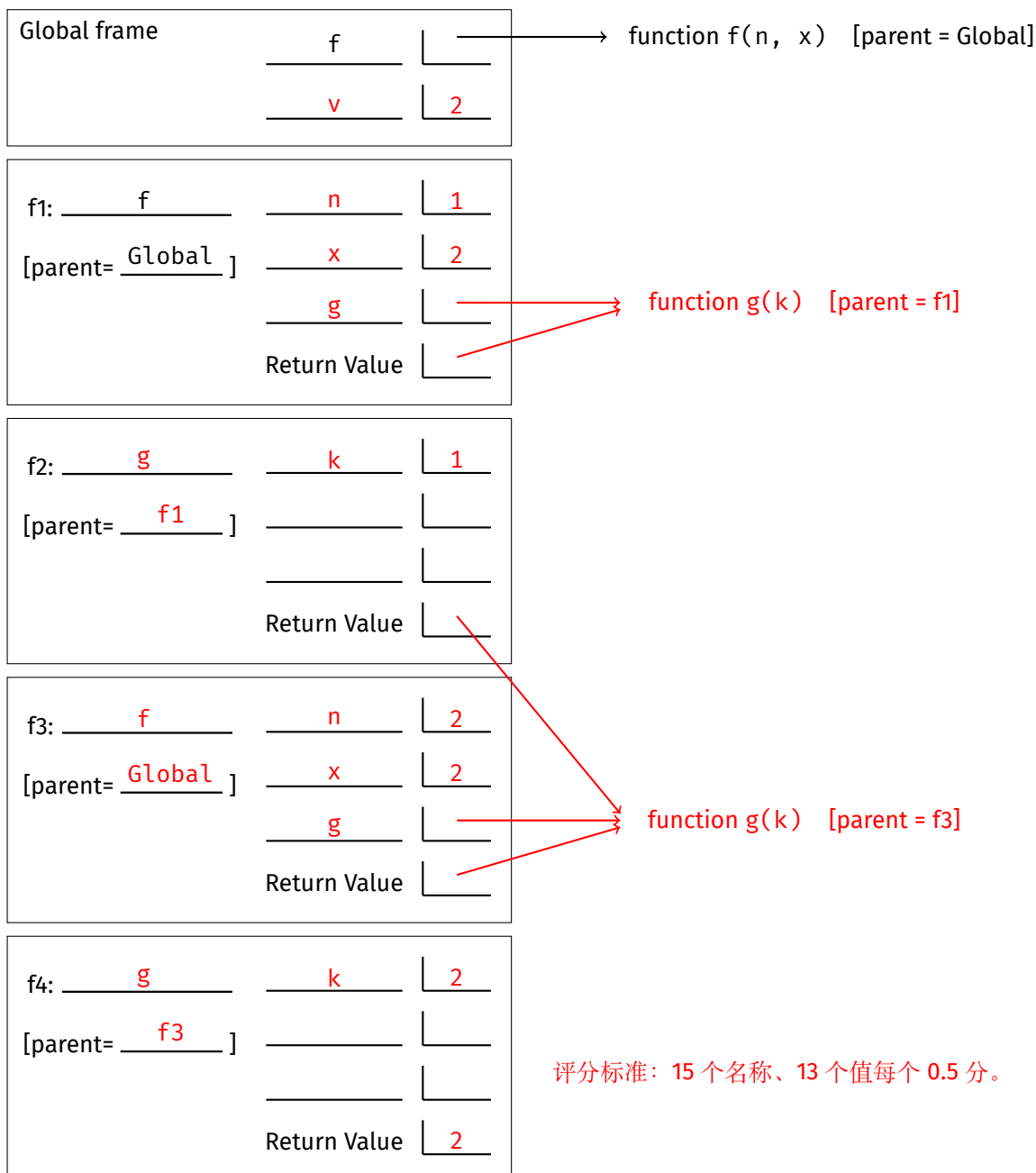
You may not need to use all of the spaces or frames.

A complete answer will:

- Add missing names and parents to all local frames.
- Add missing values created or referenced.
- Show the return value for each local frame.

```

1 def f(n, x):
2     def g(k):
3         if x == k:
4             return n
5         else:
6             return f(n + 1, x)
7     return g
8
9 v = f(1, 2)(1)(2)
    
```



3 Lists (22pts)

Which expressions in the table can get the cake? Suppose we started python3 and executed the statements below. For each expression in the table, if evaluating it returns the string "cake", write **T**. Otherwise, write **F**. (6pts)

```
1 lst = ["trap", [lambda: "cake"]]
2 lst[1].append(lst)
```

Expression	Your Answer
(lambda: "cake")()	T
(lambda x: "cake")()	F
lst[0]	F (1分)
lst[1][0]	F (1分)
lst[1][0]()	T (1分)
lst[1:][0]()	F (1分)
(lst+[lst])[2][1][0]()	T (1分)
lst.pop()[1][1][0]()	F (1分)

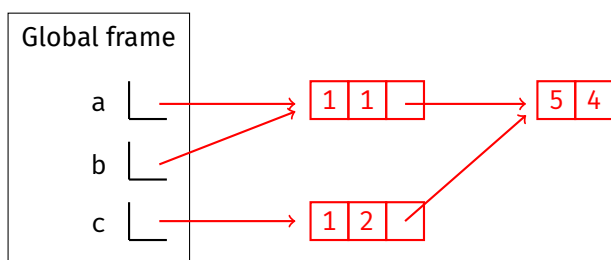
Write expressions involving the list `l` that evaluate to the following values. Each expression shall be **a single list comprehension** involving the list `l`. (4pts)

```
1 >>> l = [[1,2], [3,4], [5,6], [7,8], [9,10]]
2 >>> [x for x, y in l if y < 8] # an example for you
3 >>> [1, 3, 5]
4 >>> [[11 - x, 11 - y] for x, y in l] (2分)
5 [[10,9], [8,7], [6,5], [4,3], [2,1]]
6 >>> [2 * x for x in l[0]] (2分)
7 [2,4]
```

Draw the environment diagrams for the following two pieces of code.

Diagram #1 (6pts)

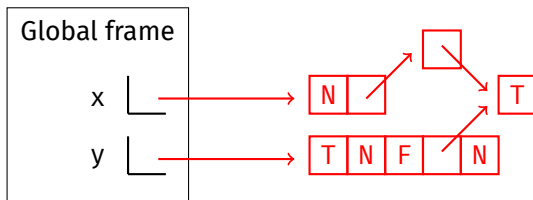
```
1 a = [1, 2, [3, 4]]
2 b = a
3 c = a[:]
4 a[1] = b[0]
5 c[2][0] = 5
```



评分标准：每个 list 正确得 2 分。

Diagram #2 (6pts)

```
1 x = [False, True, False]
2 y = []
3 for _ in range(5):
4     if x[0]:
5         x = [x.append([x[0]])] + x[1:]
6     else:
7         x = x[1:]
8     y.append(x[0])
```



评分标准：每个 **list** 正确得 3 分。True/False/None 写全更好（排版太难，答案仅供参考）。

4 Higher-Order Functions (12pts)

The successor function S is a function that takes a natural number n and returns $1 + n$. Implement a Python function s to represent S . (1pt)

```
1 def s(n):
2     """ Successor function, returns 1 + n
3     >>> list([s(i) for i in range(5)])
4     [1, 2, 3, 4, 5]
5     """
6     return 1 + n (1分)
```

A k -arity constant function C_n^k is a value, or a curried function such that:

$$C_n^0 = n$$
$$C_n^k(x_1)(x_2)(x_3) \cdots (x_k) = n$$

We use $c(n, k)$ to generate a constant function C_n^k . Implement $c(n, k)$. (4pts)

```
1 def c(n, k):
2     """ Constant function
3     >>> c(10, 0)
4     10
5     >>> c(10, 2)(1)(2)
6     10
7     """
8     if k == 0 (2分):
9         return n
10    else:
11        return lambda x: c(n, k - 1) (2分)
```

A k -arity projection function P_i^k is a curried function, such that for all $1 \leq i \leq k$:

$$P_i^k(x_1)(x_2)\dots(x_k) = x_i$$

We use $p(i, k)$ to generate a projection function P_n^k . Implement $p(i, k)$. (5pts)

Hint: make use of function c .

```
1 def p(i, k):
2     """ Projection function
3     >>> p(1, 2)(1)(2)
4     1
5     >>> p(2, 3)(1)(2)(3)
6     2
7     """
8     assert 1 <= i and i <= k
9     def helper(n):
10         if i == 1 (1分) :
11             return c(n, k - 1) (2分)
12         else:
13             return p(i - 1, k - 1) (2分)
14     return helper
```

The k -arity compose function $\text{com}(k, f, g)$ is defined as:

```
1 def com(k, f, g):
2     assert k >= 1
3     if k == 1:
4         return lambda x: f(g(x))
5     else:
6         return lambda x: com(k - 1, f, g(x))
```

Try to define a function f such that

$$f(x_1)(x_2)(x_3)(x_4)(x_5) = x_3 + 1$$

Note: you should only use com , c , p , s , and numbers (i.e., 1, 2, 3, ...). (2pts)

```
1 >>> f = com(5, s, p(3, 5)) (2分)
2 >>> f(1)(2)(3)(4)(5)
3 4
4 >>> f(9)(8)(7)(6)(5)
5 8
```

5 Trees (23pts)

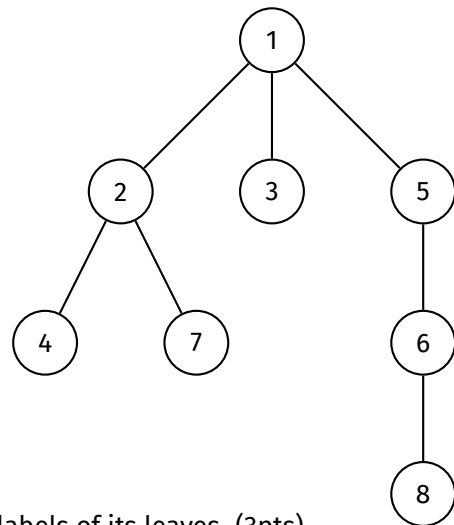
This problem is based on an assumption that all labels in a tree are different.

The following tree APIs are available for this problem:

- `tree(label, branches=[])`
- `label(tree)`
- `branches(tree)`
- `is_leaf(tree)`

Define the tree in the figure using tree constructor. (3pts)

```
t1 = tree(1, [tree(2, [tree(4), tree(7)]),  
tree(3), tree(5, [tree(6, [tree(8)])])]) (3分)
```



Implement `sum_leaves`, which takes in a tree `t` and return sum of labels of its leaves. (3pts)

```
1 def sum_leaves(t):  
2     """ Returns sum of labels of t's leaves.  
3     >>> sum_leaves(t1) # 4 + 7 + 3 + 8  
4     22  
5     """  
6     if is_leaf(t) (1分):  
7         return label(t) (1分)  
8     return sum([sum_leaves(b) for b in branches(t) (1分)])
```

The depth of a node `n` in a tree `t` is defined recursively as follows:

- If `n` is the root of `t`, its depth is 0.
- Otherwise, depth of `n` is its parent's depth plus 1.

Implement `depth`, which takes in a tree and a label. It returns the depth of the node with that label or `None`. (6pts)

```
1 def depth(t, v):  
2     """ Returns the depth of the node with given label, or None if not found.  
3     >>> depth(t1, 1), depth(t1, 3), depth(t1, 8), depth(t1, 9)  
4     (0, 1, 3, None)  
5     """  
6     if label(t) == v (1分):  
7         return 0 (1分)  
8     for b in branches(t) (1分):  
9         d = depth(b, v) (1分)  
10        if d is not None:  
11            return d + 1 (1分)  
12    return None (1分)
```

Implement `sum_lowest_leaves`, which takes in a tree `t` and returns the sum of labels of the tree's lowest leaves (lowest leaves are leaf nodes with the highest depth). (4pts)

```
1 def sum_lowest_leaves(t):
2     """ Returns the sum of labels of t's lowest leaves.
3     >>> sum_lowest_leaves(t1)
4     8
5     >>> sum_lowest_leaves(branches(t1)[0]) # 4 + 7
6     11
7     >>> sum_lowest_leaves(tree(1, [tree(2), tree(3), tree(4)]))
8     9
9     """
10    def helper(trees):
11        if all(is_leaf(t) for t in trees (两边套圆括号或中括号也正确) (1分)):
12            return sum(label(t) for t in trees) (1分)
13        return helper(sum([branches(t) for t in trees], []) (1分))
14    return helper([t] (1分))
```

Finally, implement `max_path`, which takes in a tree `t` and a positive integer `k`, and returns the highest label sum of paths in `t` with at most `k` nodes. The path may not start at the root of `t` but each step should be going downwards (from top to bottom, from root to leaf). You may assume that labels can be negative. (7pts)

```
1 def max_path(root, k):
2     """ Returns the highest label sum of paths with length no more than k.
3     >>> max_path(t1, 2) # 6 + 8
4     14
5     >>> max_path(t1, 4) # 1 + 5 + 6 + 8
6     20
7     >>> max_path(tree(1, [tree(10, [tree(20, [tree(5)]))])), 2)
8     30
9     >>> max_path(tree(5, [tree(-1, [tree(-2, [tree(-3)]))])), 2)
10    5
11    """
12    def helper(t, k, flag):
13        if k <= 0 (1分):
14            return 0
15        elif is_leaf(t) (1分):
16            return label(t)
17        max1 = max([label(t) + helper(b, k - 1, True) for b in branches(t)] + [label(t)]
(1分))
18        if flag (1分):
19            return max1 (1分)
20        max2 = max(helper(b, k, False) for b in branches(t) (1分))
21        return max(max1, max2) (1分)
22    return helper(root, k, False)
```


6 Ackermann's Function (15pts)

The following Python code computes a mathematical function called Ackermann's function. Ackermann's function takes two **non-negative integers** x and y as arguments.

```
1 def A(x, y):
2     if x == 0:
3         return y + 1
4     elif y == 0:
5         return A(x - 1, 1)
6     else:
7         return A(x - 1, A(x, y - 1))
```

What would Python display? Write your answer on the lines. (4pts)

```
1 >>> A(0, 5)
2 6 (1分)
3 >>> A(1, 0)
4 2 (1分)
5 >>> A(1, 5)
6 7 (1分)
7 >>> A(2, 3)
8 9 (1分)
```

Give concise mathematical definitions for the lambda functions in the table. (6pts)

Python Function	Mathematical Definition
<code>lambda n: 5 * n * n</code>	$5n^2$
<code>lambda n: (n - 1) * (n + 1)</code>	$n^2 - 1$
<code>lambda n: A(0, n)</code>	$n + 1$ (2分)
<code>lambda n: A(1, n)</code>	$n + 2$ (2分)
<code>lambda n: A(2, n)</code>	$2n + 3$ (2分)

Finally, fill in the blanks below to complete a different implementation of Ackermann's function. (5pts)

```
1 def A(x, y):
2     while x > 0 (或 x != 0) (1分) :
3         if y == 0:
4             y = 1
5         else:
6             y = A(x (1分) , y-1 (1分) )
7         x = x - 1 (1分)
8     return y + 1 (1分)
```

Extra Problems

- Which of the following is the correct icon (图标) of our course website and QQ group? (F) (1pt).



(A)



(B)



(C)



(D)



(E)



(F)



(G)



(H)



(I)



(J)

- Give a concise mathematical definition for the following lambda expression, where A is Ackermann's function defined in Problem 6. (1pt)

Python Function	Mathematical Definition
<code>lambda n: A(3, n)</code>	$2^{n+3} - 3$

- Are you brave enough to challenge the mass, or, stand with the mass? Here're two options for all of you. If you give up this problem or give invalid choice, you are not in this game and receive zero (0) extra points.
 - Mark the choice to "Go alone" and write a positive integer in the blank below. The *one* student who writes *the smallest, unique positive integer* will receive *one* (1) extra points but only if fewer than 98% of students choose the next option (in this case, you fail your rebellion and get zero (0) points).
 - Mark the choice to "Cooperate". If at least 98% of students choose this option, you suppress the rebellion successfully and will share the cake (*one* (1) extra point) with all students who chose this option. Or, your cooperation fails and get zero (0) extra points.

Show your choice, it's up to you.

☐ Go alone. _____

☐ Cooperate.